Assignment No 2

Aim: Apply the clustering techniques using unsupervised learning.

Theory:

Unsupervised Learning is a type of machine learning in which the model learns patterns and structures from data without using predefined output labels. Unlike supervised learning, the dataset does not contain a target variable that tells the model the correct answer. The main purpose of unsupervised learning is to discover hidden patterns, relationships, or groups within the data.

Clustering is one of the most commonly used techniques in unsupervised learning. It is the process of dividing a dataset into different groups called **clusters**, such that data points within the same cluster are more similar to each other than to data points belonging to other clusters.

For example, in a customer dataset, customers can be grouped based on their **age, income, purchase frequency, and spending amount**. Customers with similar characteristics can be placed into the same cluster. This can help businesses understand different types of customers and create suitable marketing strategies.

Common Clustering Techniques

1. **K-Means Clustering:**
   K-Means is one of the most widely used clustering algorithms. It divides the dataset into a predefined number of clusters represented by **K**. The algorithm assigns each data point to the nearest cluster centroid and repeatedly updates the centroids until the clusters become stable.

2. **Hierarchical Clustering:**
   Hierarchical clustering creates a tree-like structure called a **dendrogram**. It can be performed by either combining smaller clusters into larger ones (**agglomerative**) or dividing a large cluster into smaller clusters (**divisive**).

3. **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):**
   DBSCAN groups data points based on their density. It can identify clusters of different shapes and can also detect **noise or outliers**. Unlike K-Means, it does not require the number of clusters to be specified in advance.

K-Means Clustering

The basic steps involved in K-Means clustering are:

1. Select the number of clusters **K**.

2. Initialize K cluster centroids.

3. Calculate the distance between each data point and the centroids.

4. Assign each data point to its nearest centroid.
5. Recalculate the centroid of each cluster.
6. Repeat the assignment and centroid calculation steps.
7. Stop when the centroids or cluster assignments no longer change significantly.

The commonly used distance measure in K-Means is **Euclidean distance**:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Selection of Number of Clusters

The **Elbow Method** is commonly used to determine a suitable value of K. It calculates the within-cluster sum of squares for different values of K. The point where adding another cluster provides relatively little improvement is known as the **elbow point**.

Another technique is the **Silhouette Score**, which measures how well each data point fits within its assigned cluster. A higher silhouette score generally indicates better-defined clusters.

Importance of Clustering

Clustering is useful in many real-world applications, such as:

- **Customer segmentation** based on purchasing behavior.
- **Market segmentation** for targeted advertising.
- **Image segmentation** in computer vision.
- **Document and news categorization**.
- **Anomaly and fraud detection**.
- **Recommendation systems**.
- **Grouping students** based on academic performance.
- **Biological data analysis**.

Before applying clustering algorithms, data preprocessing is often required. Numerical features may need to be **scaled or standardized** because variables with larger numerical ranges can disproportionately influence distance-based algorithms.

Thus, clustering is an important unsupervised learning technique that helps discover natural groups and hidden patterns in unlabeled datasets. It provides useful insights from data without requiring predefined output classes.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from scipy.cluster.hierarchy import dendrogram, linkage

data = {
    'Student': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace', 'Hannah'],
    'Math': [85, 78, 92, 70, 65, 88, 55, 60],
    'Science': [80, 75, 90, 72, 60, 85, 50, 58]
}
df = pd.DataFrame(data)
print("Original Data:")
print(df)

X = df[['Math', 'Science']]

kmeans = KMeans(n_clusters=2, random_state=0)
df['KMeans_Cluster'] = kmeans.fit_predict(X)
print("\nK-Means Clusters:")
print(df[['Student', 'KMeans_Cluster']])

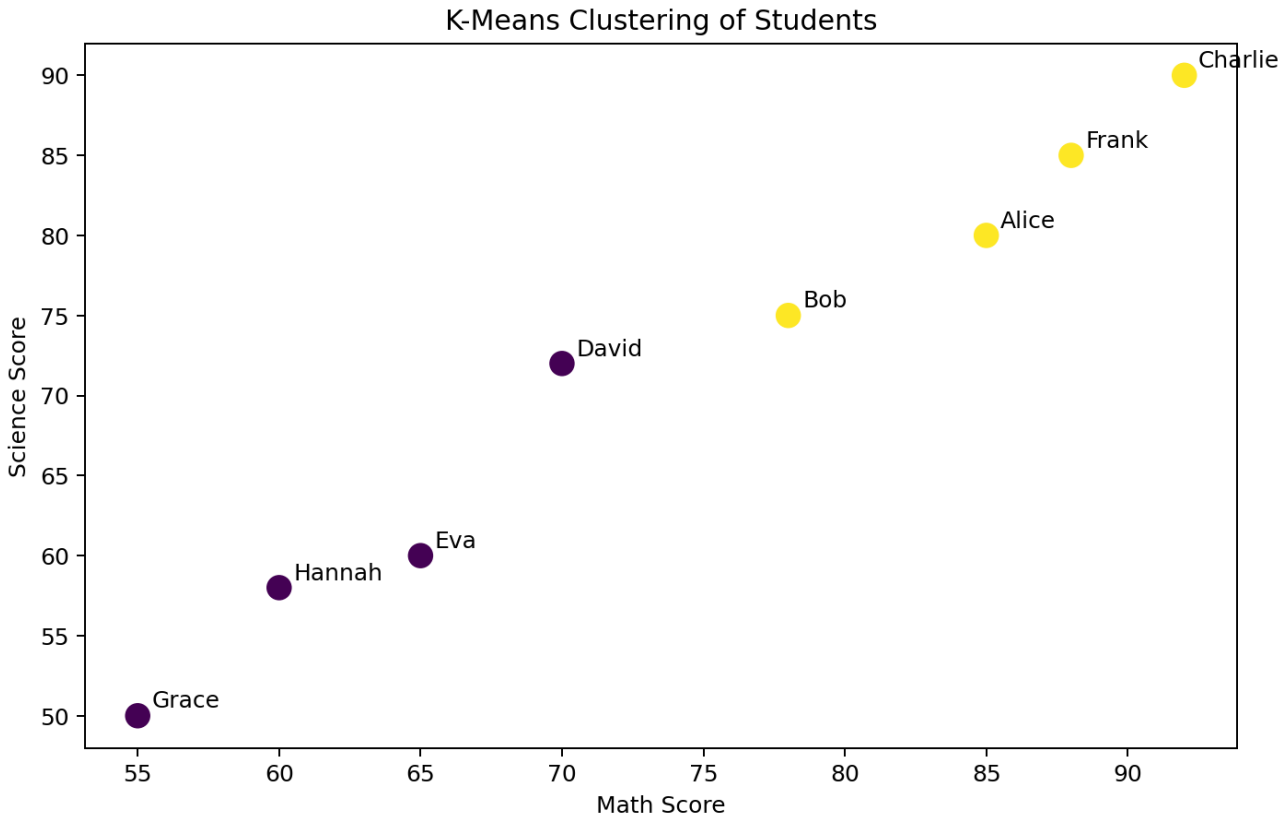
plt.figure(figsize=(6,4))
plt.scatter(df['Math'], df['Science'], c=df['KMeans_Cluster'], cmap='viridis', s=100)
plt.title('K-Means Clustering of Students')
plt.xlabel('Math Score')
plt.ylabel('Science Score')
for i, txt in enumerate(df['Student']):
    plt.annotate(txt, (df['Math'][i]+0.5, df['Science'][i]+0.5))
plt.show()

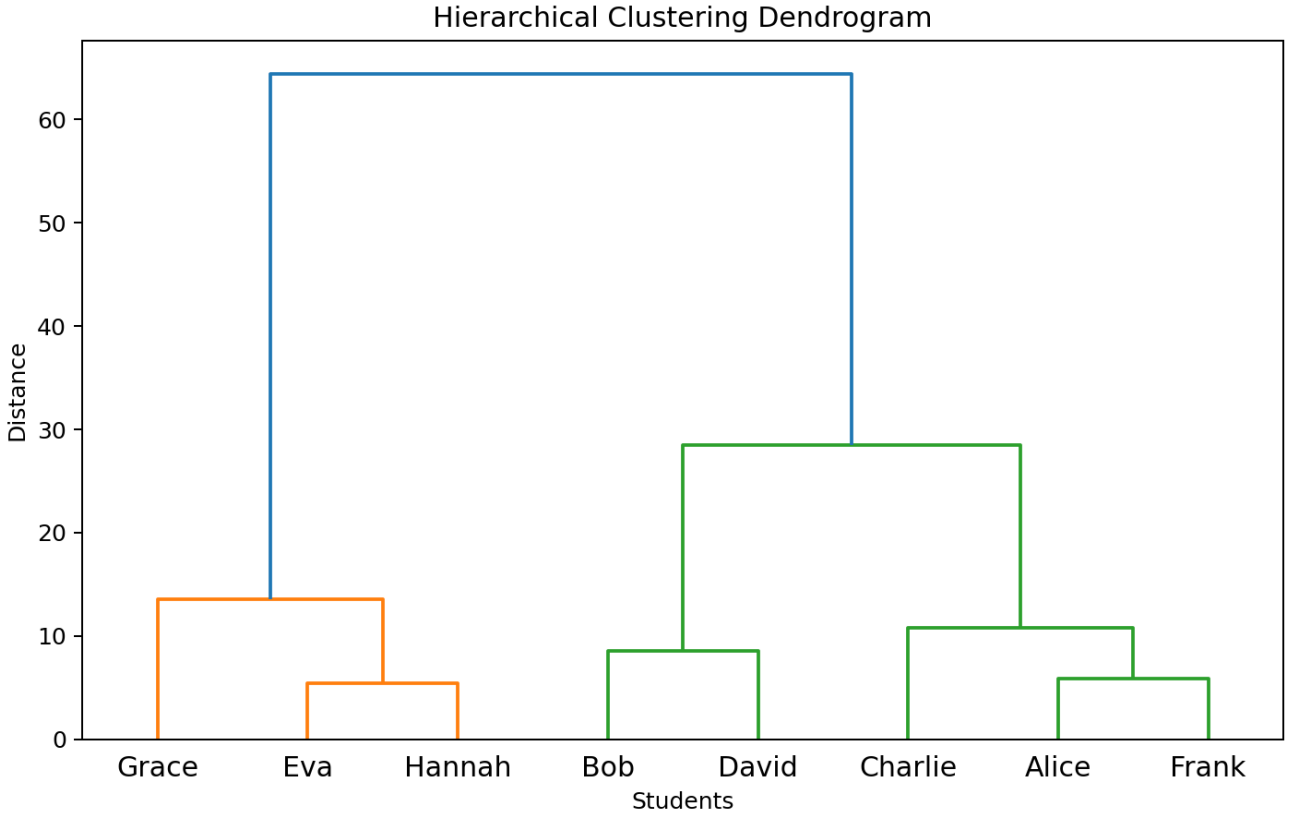
linked = linkage(X, method='ward')
plt.figure(figsize=(8,4))
dendrogram(linked, labels=df['Student'].values)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Students')
plt.ylabel('Distance')
plt.show()

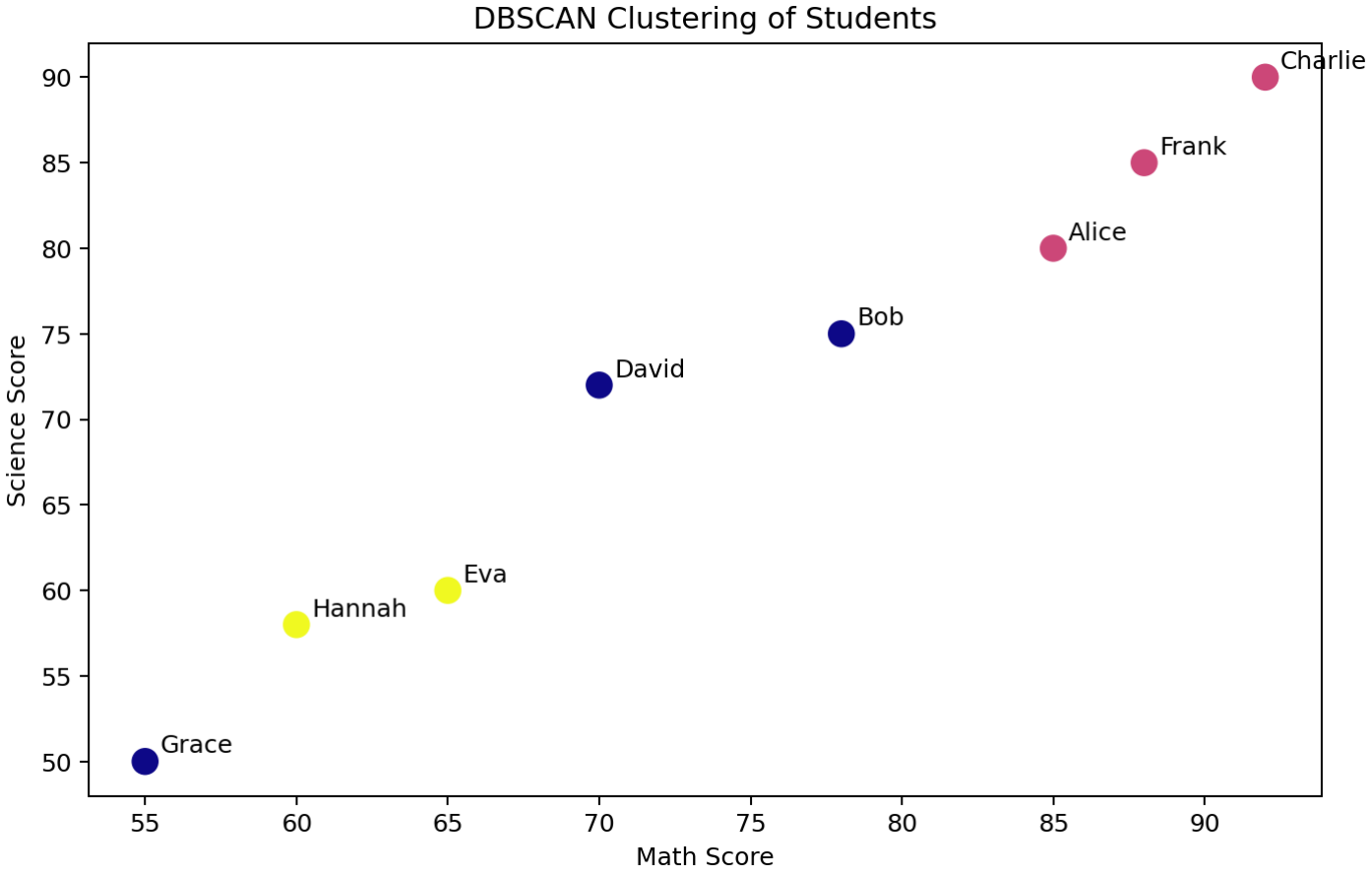
dbscan = DBSCAN(eps=7, min_samples=2)
df['DBSCAN_Cluster'] = dbscan.fit_predict(X)
print("\nDBSCAN Clusters:")
print(df[['Student', 'DBSCAN_Cluster']])

plt.figure(figsize=(6,4))
plt.scatter(df['Math'], df['Science'], c=df['DBSCAN_Cluster'], cmap='plasma', s=100)
plt.title('DBSCAN Clustering of Students')
plt.xlabel('Math Score')
plt.ylabel('Science Score')
for i, txt in enumerate(df['Student']):
    plt.annotate(txt, (df['Math'][i]+0.5, df['Science'][i]+0.5))
plt.show()
```

OUTPUT:


K-Means Clustering of Students
Science Score
Math Score
Charlie
Frank
Alice
Bob
David
Eva
Hannah
Grace
90
85
80
75
70
65
60
55
50
55
60
65
70
75
80
85
90


Hierarchical Clustering Dendrogram
Distance
Students
60
50
40
30
20
10
0
Grace
Eva
Hannah
Bob
David
Charlie
Alice
Frank


DBSCAN Clustering of Students
Science Score
Math Score
90
85
80
75
70
65
60
55
50
55
60
65
70
75
80
85
90
Charlie
Frank
Alice
Bob
David
Eva
Hannah
Grace